

Angular HandsOn 9: State Management with NgRx

HANDS-ON 9 [Advanced]

State Management ? NgRx Store, Actions, Reducers, Effects & Selectors

Topics Covered:

When to Use NgRx vs Services[Check] Actions & Action Creators[Check]

Reducers & Immutable State[Check] Selectors & Memoisation[Check]

NgRx Effects for Async Operations[Check] RxJS Observables & Operators in NgRx [Check]

Context

As Angular applications grow, managing shared state in services becomes hard to maintain and debug. NgRx brings Redux-pattern state management to Angular with powerful developer tooling. You will migrate the course and enrollment state to NgRx.

INS

TAL

L

`npm install @ngrx/store @ngrx/effects @ngrx/entity @ngrx/store-devtools` ? also install Redux

DevTools Chrome extension

Task 1: Set Up NgRx Store and Define Course State

Goal: Configure the NgRx store and create actions, reducer, and selectors for courses.

Steps:

92. Import `StoreModule.forRoot({})` in `AppModule` (or `provideStore()` in `app.config.ts`). Import `StoreDevtoolsModule.instrument({ maxAge: 25 })` for debugging.

93. Create the course state files inside `store/course/`: `course.actions.ts`, `course.reducer.ts`, `course.selectors.ts`. In `course.actions.ts`, define: `loadCourses = createAction('[Course] Load Courses')`, `loadCoursesSuccess = createAction('[Course] Load Courses Success', props<{ courses: Course[] }>())`, `loadCoursesFailure = createAction('[Course] Load Courses Failure', props<{ error: string }>())`.

94. In `course.reducer.ts`, define the state interface: `{ courses: Course[], loading: boolean, error: string | null }`. Create the reducer using `createReducer` with `on()` handlers for all three actions.

95. Register the reducer: `StoreModule.forFeature('course', courseReducer)` (or `provideState` in standalone). In `course.selectors.ts`, create: `selectCourseState`, `selectAllCourses`, `selectCoursesLoading`, `selectCoursesError` using `createFeatureSelector` and `createSelector`.

96. In `CourseListComponent`, replace the service subscription with: `this.courses$ = this.store.select(selectAllCourses)`. Dispatch the load action in `ngOnInit`: `this.store.dispatch(loadCourses())`. Render using the async pipe: `<app-course-card *ngFor='let c of courses$ | async'>`.

Hint:

- The `[Course]` prefix in action type strings is a convention that groups actions by feature ? it makes the

Angular HandsOn 9: State Management with NgRx

Redux DevTools timeline readable: you can filter by '[Course]' t- see only course-related actions.

- Selectors are memoised ? they only recompute when their input selectors change. This is NgRx's key performance optimisation.

Expected Outcome: Redux DevTools shows the loadCourses action dispatched on page load. State tree shows courses, loading, and error. Courses render via the async pipe.

Task 2: NgRx Effects for HTTP and Enrollment State

Goal: Use NgRx Effects t- handle async API calls and add enrollment state t- the store.

Steps:

97. Create course.effects.ts. Define a loadCourses\$ effect: `createEffect(() =>`

`this.actions$.pipe(ofType(loadCourses), switchMap(() =>`

`this.courseService.getCourses().pipe(map(courses => loadCoursesSuccess({ courses }))),`

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book

`catchError(error => of(loadCoursesFailure({ error: error.message })))))). Register CourseEffects in EffectsModule.forRoot([CourseEffects]) or provideEffects.`

98. Verify the full flow: dispatching loadCourses ? Effect fires HTTP ? loadCoursesSuccess dispatched ? reducer updates state ? selector emits new value ? component re-renders. Trace each step in Redux DevTools.

99. Add enrollment state: create store/enrollment/ with actions (enrollInCourse, unenrollFromCourse, setEnrolledCourses), a reducer managing enrolledCourseIds: number[], and selectors (selectEnrolledIds, selectEnrolledCourses using a cross-slice selector combining course and enrollment state).

100. Dispatch enrollInCourse({ courseId: id }) from CourseCardComponent when Enroll is clicked. Use selectEnrolledIds t- show/hide the Unenroll button: `{{ (enrolledIds$ | async)?.includes(course.id) ? 'Unenroll' : 'Enroll' }}`.

Hint:

- Effects are the only place in NgRx where side effects (HTTP calls, navigation, localStorage) should happen ? reducers must remain pure functions.

- Cross-slice selectors (combining course and enrollment state) are a powerful NgRx pattern ? use createSelector with multiple input selectors t- derive joined data without duplicating state.

Expected Outcome: Redux DevTools shows the complete action ? effect ? success flow. Enrollment state updates correctly. The async pipe renders enrolled/unenrolled state per card.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Angular (v20.0) | Hands-On Exercise Book